



Boletín 03 Inicial — Ejercicios Prácticos

Ejercicios fundamentales de bucles, excepciones básicas, arrays unidimensionales y modularidad con métodos en Java.

Bienvenido al boletín de nivel inicial de la **Unidad 3: Excepciones, bucles, arrays y métodos**.

Estos 12 ejercicios están diseñados para asentar la lógica estructurada, el uso de las estructuras iterativas (`while`, `do-while`, `for`), la gestión elemental de excepciones de entrada con `Scanner`, la manipulación de vectores y la definición de métodos estáticos.

Consejo de Estudio

Intenta resolver cada ejercicio por tu cuenta en IntelliJ IDEA antes de consultar el boletín resuelto. Analiza las posibles entradas erróneas del usuario para proteger tu programa con bloques `try-catch`.

Ejercicio 1: Tabla de Multiplicar Segura

Enunciado:

Solicita al usuario por teclado un número entero entre 1 y 10. Muestra su tabla de multiplicar completa del 1 al 10 con formato tabulado. Si el usuario introduce letras o un número fuera del rango `[1, 10]`, captura la excepción correspondiente o repite la petición hasta que el dato sea válido.

Pistas: Utiliza un bucle `while` o `do-while`, un bloque `try-catch` capturando `InputMismatchException` y limpia el buffer con `scanner.nextLine()`.

Ejercicio 2: Contador de Dígitos y Suma de Cifras

Enunciado:

Pide al usuario un número entero positivo (de tipo `long`). Empleando exclusivamente divisiones sucesivas (`/ 10`) y el operador módulo (`% 10`) dentro de un bucle `while`, calcula cuántos dígitos tiene el número y cuánto suman todas sus cifras.

Ejemplo: Para `4523`, el programa debe indicar: 4 dígitos y suma total 14.

Ejercicio 3: Menú Interactivo de Gestión de Saldo

Enunciado:

Diseña un simulador de terminal de punto de venta. El programa parte de un saldo inicial de 100.0€. Presenta un menú en consola con las siguientes opciones:

1. Consultar saldo disponible.
2. Ingresar dinero (no se admiten cantidades negativas ni nulas).
3. Retirar dinero (no se puede retirar más del saldo disponible).
4. Salir.

Requisitos: Usa un bucle `do-while` para mantener el menú activo hasta seleccionar la opción 4.

Ejercicio 4: Dibujando Patrones con Bucles Anidados

Enunciado:

Pide al usuario un número entero `N` (altura entre 2 y 15). Empleando dos bucles `for` anidados, dibuja en consola un triángulo rectángulo de asteriscos alineado a la izquierda:

```
*
**
***
****
*****
```

Ampliación voluntaria: Dibuja el triángulo invertido hacia abajo.

Ejercicio 5: La Conjetura de Collatz

Enunciado:

Pide un número entero `n > 0`. Si `n` es par, se divide entre 2 (`n / 2`). Si `n` es impar, se multiplica por 3 y se le suma 1 (`3n + 1`). Repite la operación hasta que `n` llegue a 1. El programa debe imprimir toda la secuencia en una sola línea separada por flechas `→` y mostrar al final el número total de pasos requeridos.

Ejercicio 6: Análisis de Temperaturas en un Vector

Enunciado:

Declara un array de `double` de tamaño 7 que represente las temperaturas medias de una semana. Solicita por teclado las 7 temperaturas protegiendo la lectura de fallos. A continuación, calcula e imprime:

- La temperatura máxima registrada y el día en que ocurrió (índice 0 = Lunes, etc.).
- La temperatura mínima registrada.
- La temperatura media de la semana formateada con 2 decimales.

Ejercicio 7: Registro de Calificaciones y Estadística de Grupo

Enunciado:

Crea un programa que almacene las calificaciones (entre 0.0 y 10.0) de un grupo de 10 alumnos en un array `double[]`. Recorre el array mediante un bucle mejorado `for-each` y determina:

- Cuántos alumnos han obtenido sobresaliente (nota ≥ 9.0).
- Cuántos han aprobado (nota ≥ 5.0).
- Cuántos han suspendido (nota < 5.0).

Ejercicio 8: Inversión In-Place de un Vector

Enunciado:

Escribe un programa que cree un array con los números `{10, 20, 30, 40, 50, 60, 70}`. Sin utilizar un segundo array auxiliar, invierte los elementos directamente en memoria intercambiando los extremos con la técnica de dos punteros (`izquierda` y `derecha`). Imprime el array resultante con `Arrays.toString()`.

Ejercicio 9: Contabilizador de Vocales y Consonantes

Enunciado:

Pide una frase por consola. Conviértela a minúsculas y, recorriéndola carácter a carácter con `charAt()`, cuenta cuántas vocales (`a, e, i, o, u`), cuántas consonantes y cuántos caracteres no alfabéticos (espacios, números, puntuación) contiene la cadena.

Ejercicio 10: Métodos Sobrecargados para el Cálculo de Áreas

Enunciado:

En una clase con métodos modulares, define tres métodos sobrecargados llamados `calcularArea`:

1. `public static double calcularArea(double lado)` -> Para el área de un cuadrado.
2. `public static double calcularArea(double base, double altura)` -> Para un rectángulo.
3. `public static double calcularArea(double radio, boolean esCirculo)` -> Para el área de un círculo ($\pi * \text{radio}^2$).

Valida que ninguna dimensión pueda ser menor o igual a cero lanzando `IllegalArgumentException`.

Ejercicio 11: Potencia Recursiva

Enunciado:

Implementa un método recursivo con la firma `public static long calcularPotencia(int base, int exponente)`.

- Caso base: Cualquier número elevado a 0 es 1 (`exponente == 0 → 1`).
- Caso recursivo: `base * calcularPotencia(base, exponente - 1)`.
- Control de precondition: Si el exponente es negativo, lanza una `IllegalArgumentException`.

Ejercicio 12: Validador de Código Postal Español con Regex

Enunciado:

Escribe un método `public static boolean esCodigoPostalValido(String cp)` que verifique mediante una expresión regular si una cadena cumple con el formato oficial de un código postal en España: exactamente 5 dígitos numéricos, donde los dos primeros dígitos deben estar comprendidos entre 01 y 52 (las 52 provincias españolas).

¿Necesitas comprobar tus soluciones?

Puedes ver todas las soluciones completas y explicadas en el [Boletín 3.1 Resuelto](#).